

Chapter 5

Network Layer

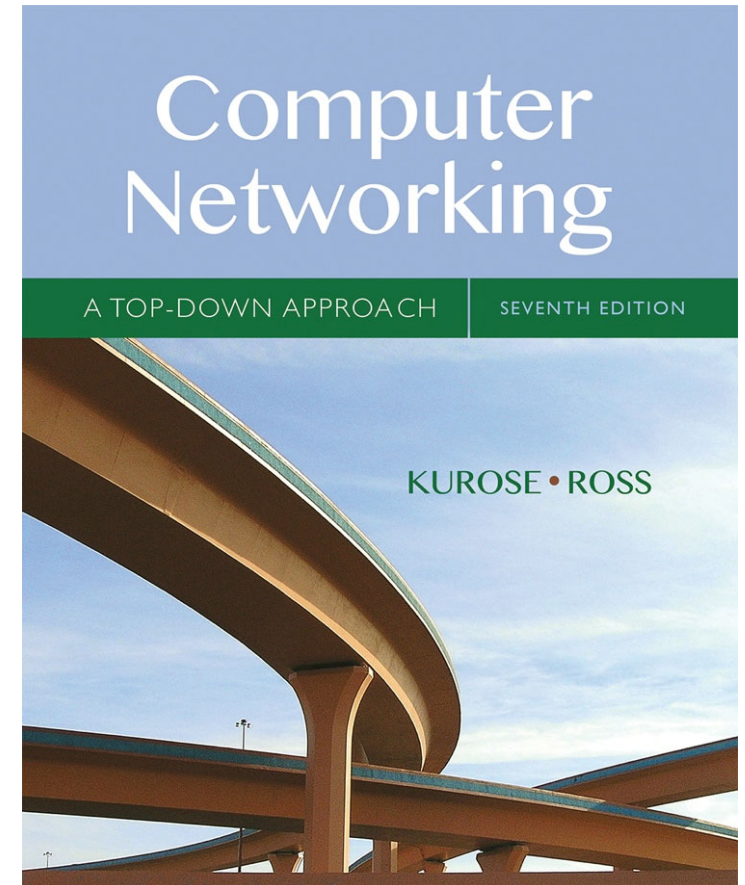
A note on the use of these Powerpoint slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

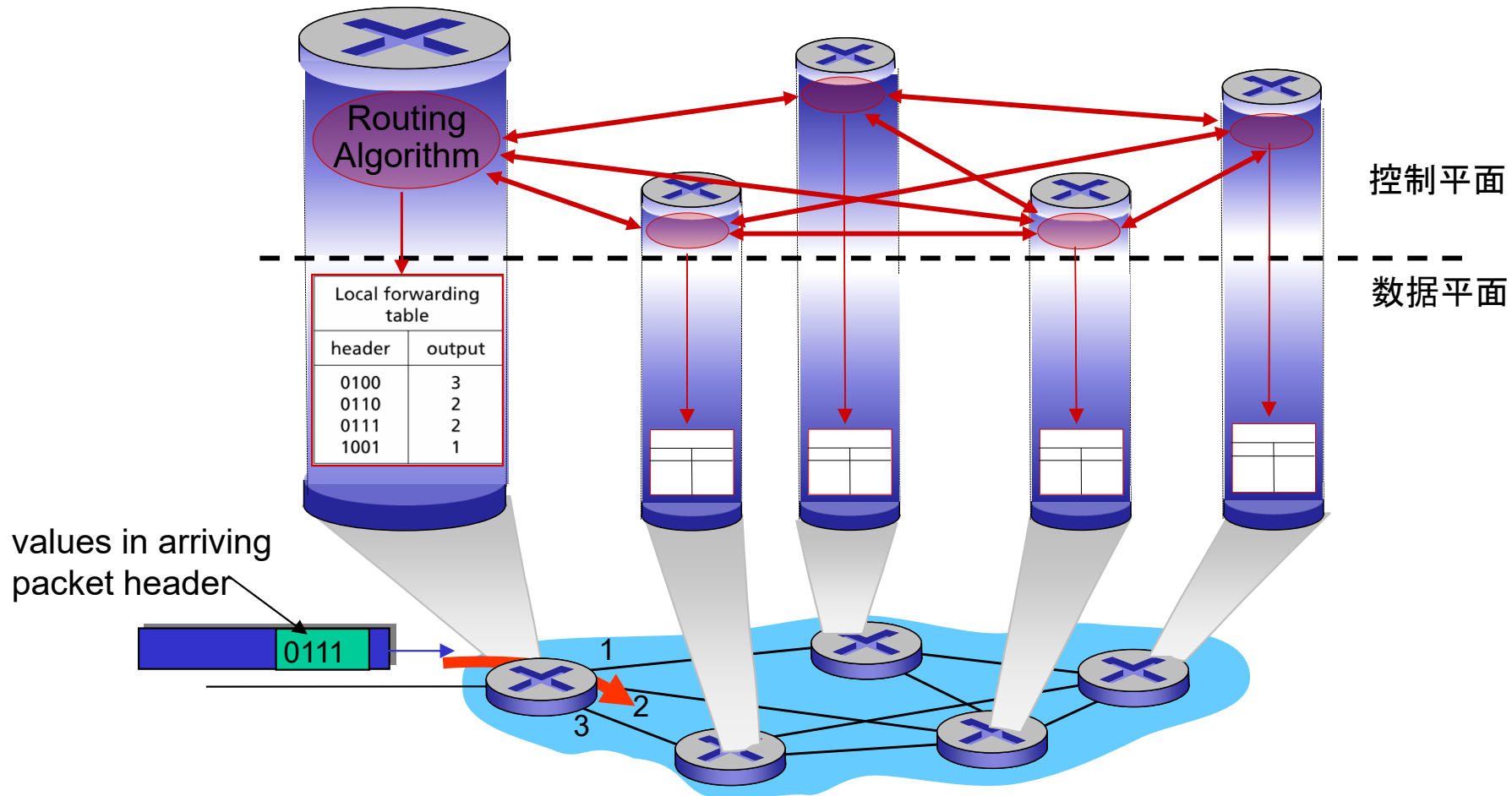
All material copyright 1996-2016
J.F Kurose and K.W. Ross, All Rights Reserved



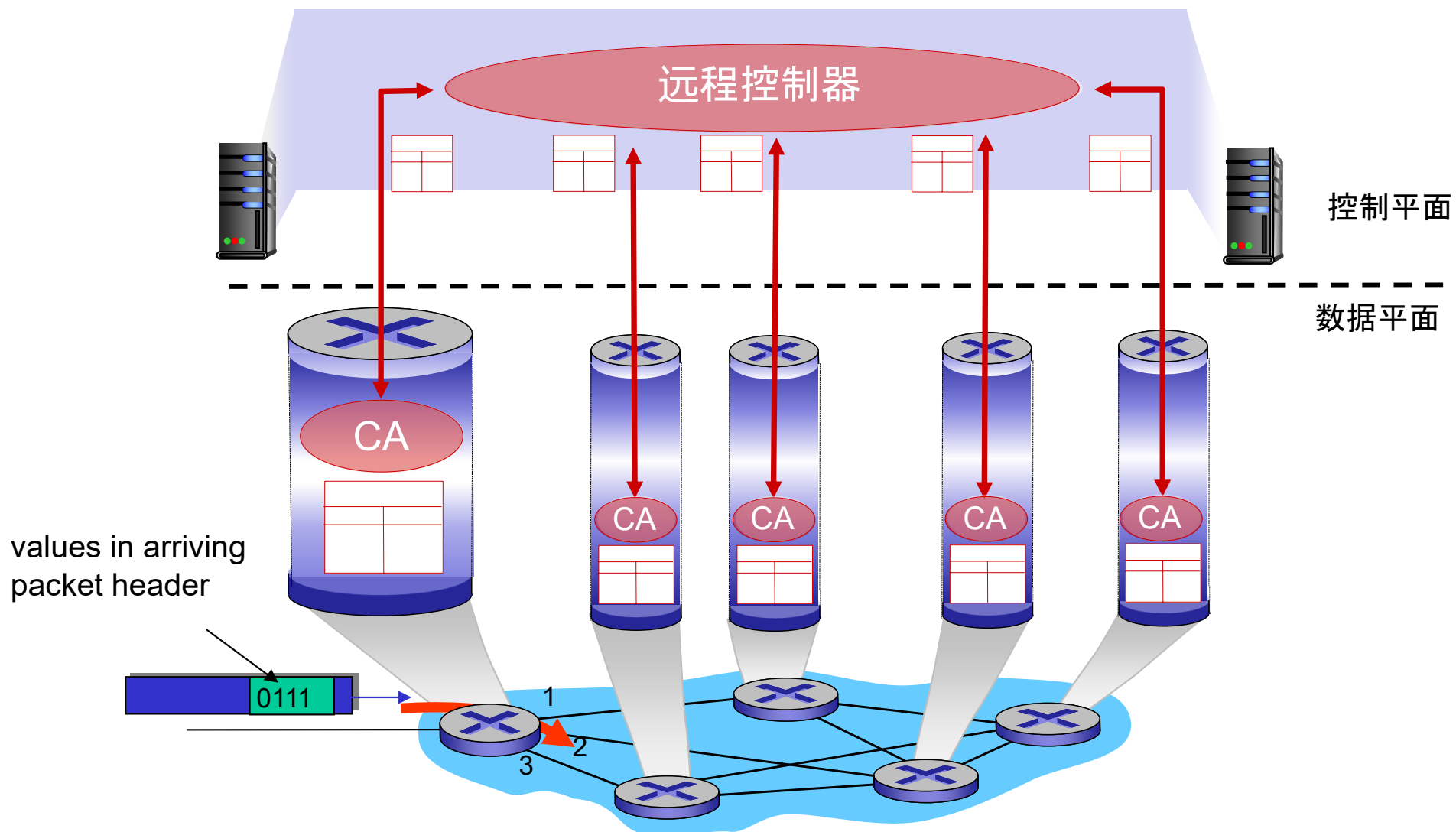
Computer Networking: A Top Down Approach

5.1 控制平面概述

每个路由有独立的路由算法，通过路由器交互形成转发表



- **远程控制器**计算并分发转发表
- 可能实现在具有高可靠和冗余的远程数据中心中，可能由ISP或某些第三方管理
- 路由器和远程控制器通过交换包含**转发表**和**其他路由选择信息**的报文



5.2 路由选择算法

确定一个分组从源路由器到目的路由器所经**路径的算法**

即在给定的一组路由器以及连接路由器的链路中，找到一条从源路由器到目的路由器的“好”路径

r “好”路径：具有“**最低开销**”的路径

由许多因素决定，如链路长度、传播时延、价格等

网络的抽象图模型

用“**节点图**”表示网络的结构。

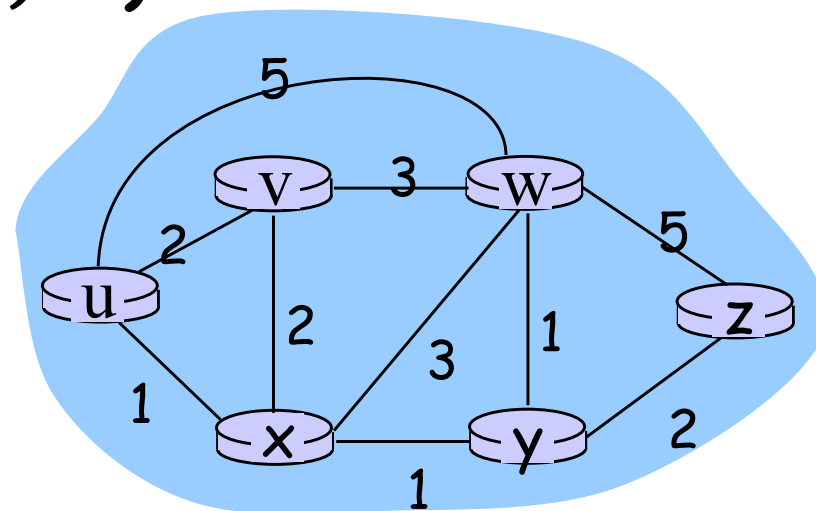
图 $G = (N, E)$ 表示 N 个节点和 E 条边的集合，每条边是来自 N 的一对节点。

✓ **节点**：表示路由器(做出分组转发判决的点)。

如 u, v, w, x, y, z 。

✓ **边**：连接节点的线段，表示路由器之间的物理链路。

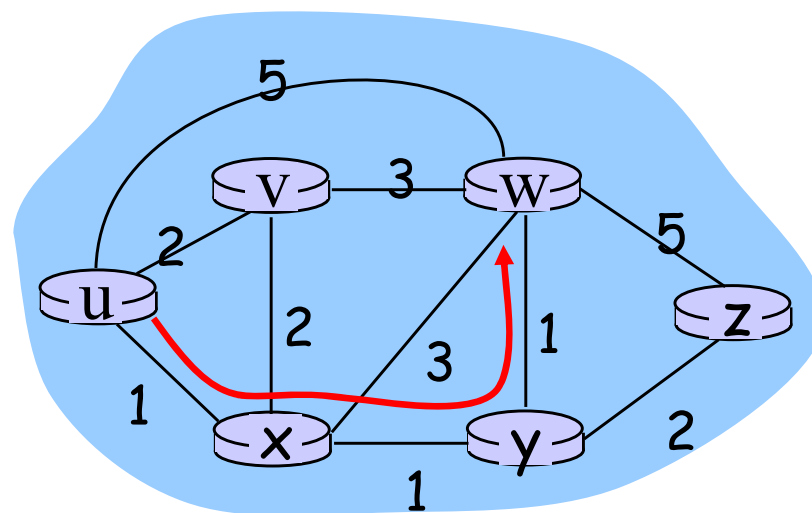
如 (u, v) 、 (u, x) 、 (u, w) 、...



例

如图，源节点u和目的节点w之间的最低费用路径：

(u, x, y, w) ，费用为3。



选路算法分类

根据算法是全局性还是分散性划分

根据算法是静态还是动态划分

根据对负载是否敏感划分

常用的两种选路算法：

- ✓ 集中式路由选择算法：动态全局链路状态算法
- ✓ 分散式路由选择算法：动态分散式距离向量算法

5.2.1 链路状态路由选择算法LS

5.2.2 距离向量路由选择算法DV

集中式路由选择算法

用完整的、全局性的网络信息来计算最低费用路径。
即以所有节点之间的连通性及所有链路的费用为条件。

- ✓ 在开始计算前，以某种方式获得这些信息。
- ✓ 可在单个位置计算，也可在多个位置上复制。
- ✓ 拥有连通性和链路费用的完整信息。

链路状态算法LS：必须知道网络中每条链路的费用。

分散式路由选择算法

以**迭代的、分布式的方式**计算最低费用路径。

- ✓ 节点**只有与其直接相连链路的信息**：不需拥有所有网络链路费用的完整信息。
- ✓ 通过**迭代计算**过程并与相邻节点(邻居节点)交换信息
- ✓ **逐步计算**出到达某目的节点或一组目的节点的最低费用路径。

距离向量算法DV：每个节点维护到网络中所有其他节点的费用（距离）的估计向量。

✓ **静态选路算法:**

路由确定后基本不再变化。当人工干预调整时，可能有一些变化。

✓ **动态选路算法:**

当网络的流量负载或拓扑发生变化时，改变路径。

可以周期性地或直接地响应拓扑或链路费用的变化。

易受选路循环、路由振荡之类问题的影响。

5.2.1 链路状态选路算法LS

- r 前提条件：已知网络拓扑和所有链路费用，作为算法的输入。
- r 获取方法：每个节点向网络中广播链路状态分组（含有它所连接的链路费用）。

由链路状态广播算法实现。最终使所有节点都有一个相同且完整的网络视图。

- r 每个节点都可以运行链路状态算法并计算出最低费用路径集。

Dijkstra最低费用路径算法（最短通路算法）

Edsger Wybe Dijkstra (Dutch: ['etsxər 'vibə 'deɪkstra])

✓ 计算从某节点（**源节点，如 u** ）到网络中所有其他节点的最低费用路径。

✓ 是一种**迭代算法**，即：

经第 **k** 次迭代后，可知道到 **k** 个目的节点的最低费用路径。

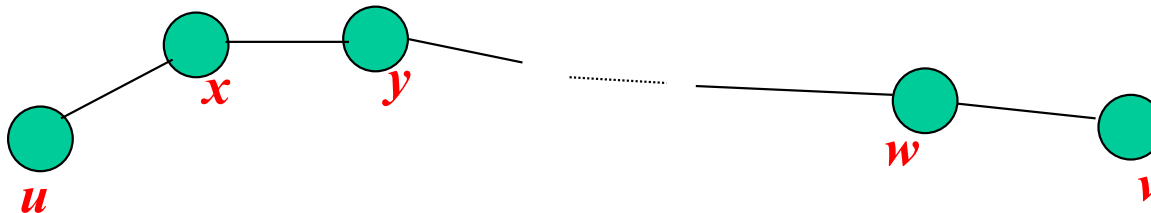
✱ **基本思想：**

以**源节点为起点**，**每次找出一个到源节点的费用最低的节点**，直到把所有的目的节点都找到为止。



符号定义:

- ✓ $c(x, y)$: 从节点 x 到 y 的链路费用;
= ∞ 如果不是直接邻居
- ✓ $D(v)$: 从源节点到目的节点 v 的当前费用 ;
- ✓ $p(v)$: 从源节点到节点 v 的路径上的前一节点(节点 v 的邻居);
- ✓ N' : 节点集合, 从源节点到这些节点的最低费用路径已被求出。



Dijkstra算法组成

- ✓ 由一个**初始化步骤**和其后的**循环**组成。
- ✓ 循环执行的次数与网络中的**节点个数（除源节点）**相同。
- ✓ 结束时，算出从源节点到网络中所有其他节点的最短路径。

算法

```
1  初始化:
2   $N' = \{u\}$ 
3  对所有节点  $v$ 
4      if  $v$  临近  $u$ 
5          then  $D(v) = c(u, v)$ 
6          else  $D(v) = \infty$ 
7
8  Loop
9      找出  $w$  不在  $N'$  中使得  $D(w)$  最小
10     将  $w$  加入  $N'$ 
11     对于所有  $v$  临近  $w$  并不在  $N'$  中, 更新  $D(v)$ :
12          $D(v) = \min( D(v), D(w) + c(w, v) )$ 
13     /* 到  $v$  的新费用或是到  $v$  的老费用或到  $w$  加上从  $w$  到  $v$  的已知
        最短路费用 */
15 until 所有节点在  $N'$  中
```

算法描述

(1) 初始化 (1#~6#)

$N' = \{\text{源节点 } u\};$

对所有不在 N' 中的节点 v ，标出其 $D(v)$ 值：

- ✓ 节点 v 与源节点 u 直接相连， $D(v) = c(u, v)$
- ✓ 节点 v 与源节点 u 不直接相连， $D(v) = \infty$

```
1  初始化:
2     $N' = \{u\}$ 
3    对所有节点 $v$ 
4      if  $v$  临近  $u$ 
5        then  $D(v) = c(u, v)$ 
6      else  $D(v) = \infty$ 
```


(2) 找出一个到源节点的费用最低的节点 w ，并以此更新其它点 $D(v)$ 值

①从不在 N' 中的节点中找出一个 $D(w)$ 值最小的节点 w ，并将 w 加入到 N' 中。

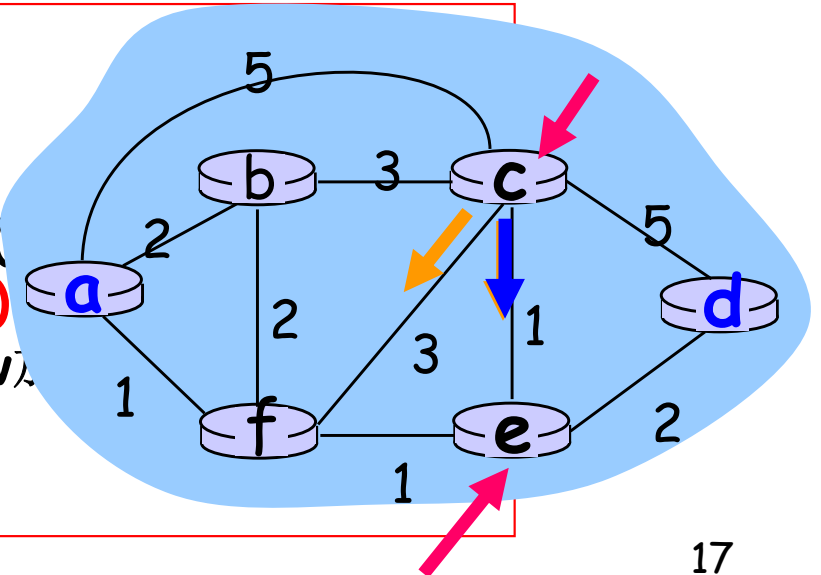
②对不在 N' 中，但与节点 w 是邻居的节点 v ，用新的值更新 $D(v)=\min[D(v), D(w)+c(w,v)]$

将节点 v 原值与节点 v 现经节点 w 到源节点的值比较，取小值。

(3) 重复步骤(2)

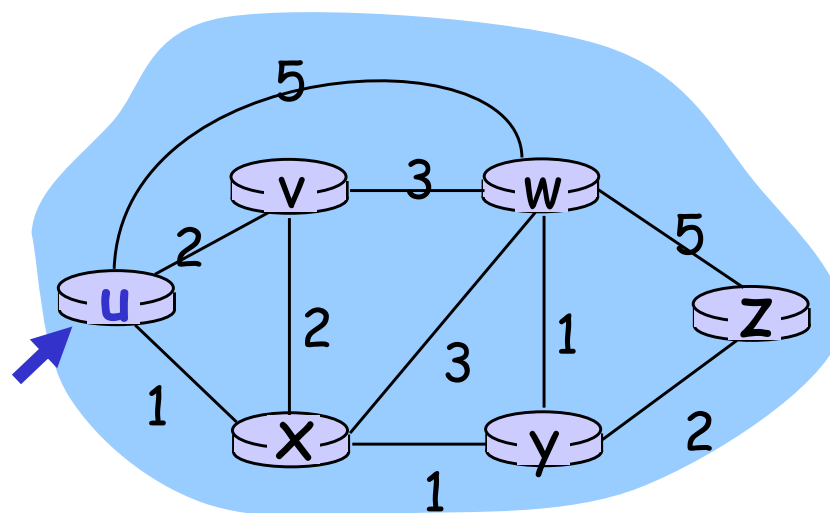
直到所有的网络节点都在 N' 中为止。

```
8  Loop
9   找出 $w$ 不在 $N'$  中使得 $D(w)$ 最小
10  将 $w$ 加入 $N'$ 
11  对于所有 $v$ 临近 $w$ 并不在 $N'$  中，更新 $D(v)$ 
12       $D(v) = \min( D(v), D(w) + c(w,v) )$ 
13  /* 到 $v$ 的新费用或是到 $v$ 的老费用或到 $w$ 
最短路费用*/
15  until 所有节点在  $N'$  中
```



例

r 计算从u到所有可能目的节点的最低费用路径。

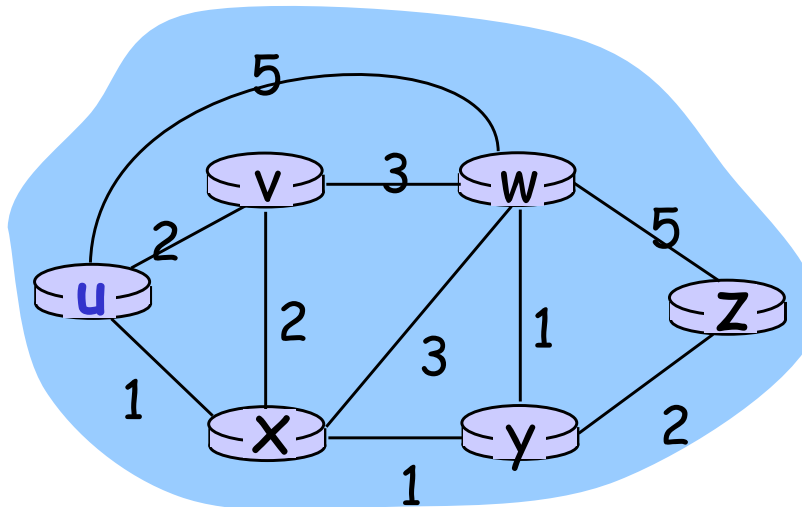


步骤	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞

$$D(w): \min(5, 1+3)=4$$

$$D(y): \min(\infty, 1+1)=2$$

步骤	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



r 默认路由 * :

- ✓ 表示所有具有相同“下一跳”的表项。即将“下一跳”相同的项合并为一项，目的节点用“*”表示。
- ✓ 优先级最低，转发分组时，当找不到对应表项时，才使用默认路由。

节点u的转发表

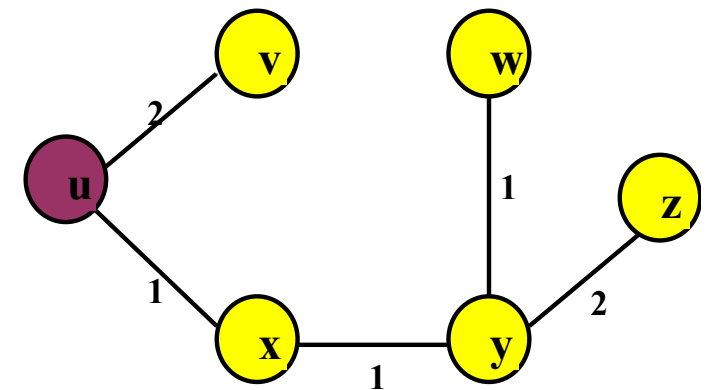
目的点 下一跳

u	—
v	v
w	x
x	x
y	x
z	x

节点u的转发表

目的点 下一跳

u	—
v	v
*	x



将网络中每一个节点作为源点，分别用上述方法计算，可以得到每一个节点的转发表。

算法的计算复杂性

- r 设 n 个节点(除源节点), 最坏情况下要经多少次计算才能找到从源节点到所有目的节点的最低费用路径?
- ✓ 第一次迭代: 搜索所有的 n 个节点以确定最低费用节点
- ✓ 第二次迭代: 检查 $n-1$ 个节点;
- ✓ 第三次: 检查 $n-2$ 个节点;

依次类推。

所有迭代中需要搜寻的节点总数为 $n(n+1)/2$ 。

- r 算法复杂性为 n 平方阶序 $O(n^2)$ 。

LS算法的缺陷

可能产生“振荡”。

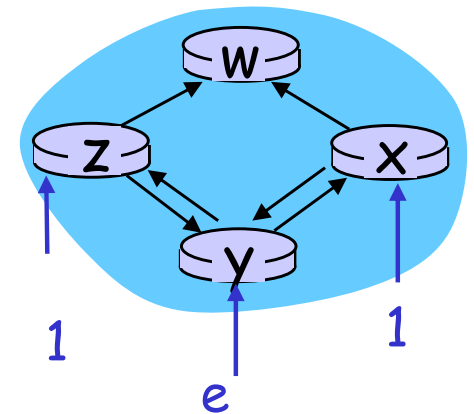
例一个简单网络拓扑。设：

- ✓ 链路费用等于链路上的负载；
- ✓ 链路费用是**非对称的**：

$c(u, v)$ 与 $c(v, u)$ 仅当在链路 (u, v) 两个方向的负载相同时才相等。

有三个同时发往**w**的注入流量：

- ✓ **节点z**：一个单元；
- ✓ **节点x**：一个单元；
- ✓ **节点y**： e 的流量。



✓ **初始:** 图(a), 其链路费用相当于承载的流量

✓ **第一次:**

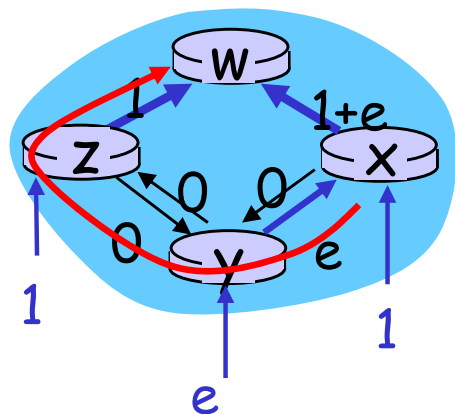
节点 x 、 y 都确定**顺时针方向**到 w 的路径费用最低, 为1。产生图(b)费用。

✓ **第二次:**

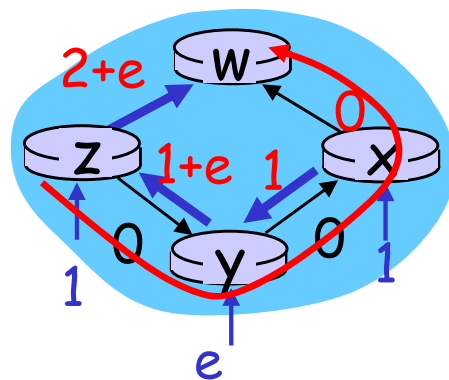
节点 x 、 y 、 z 都确定**逆时针方向**到 w 的路径费用最低, 为0。产生图(c)费用。

✓ **第三次:**

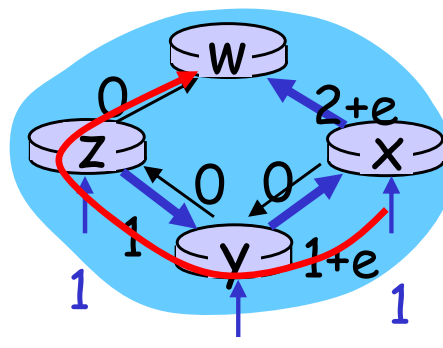
节点 x 、 y 、 z 都确定**顺时针方向**到 w 的路径费用最低, 为0。产生图(d)费用。



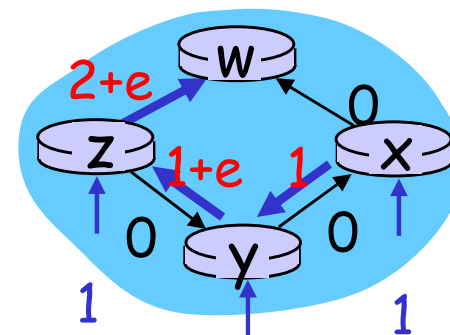
(a)



(b)



(c)



(d)

避免振荡

- ✓ 使链路费用不依赖于所承载的流量：不可行；
- ✓ 避免所有的路由器同时运行LS算法：比较合理。

即使路由器以相同周期运行LS算法，在每个节点上算法的执行时刻也不同。

r 因特网的自同步：

即使路由器初始时以同一周期但不同时刻执行算法，算法执行时刻最终会同步并保持。

采用随机时间的方法，即每台路由器随机发送链路通告。

5.2.2 距离向量选路DV算法

是一种迭代的、异步的和分布式的算法。

- ✓ 分布式：每个节点都**从其直接相连邻居接收信息**，进行计算，再将计算结果分发给邻居。
- ✓ 迭代：计算过程一直持续到**邻居之间无更多信息交换为止**。
- ✓ 自我终结：算法能自行停止。
- ✓ 异步：不要求所有节点相互之间步伐一致地操作。

最低费用表示

$d_x(y)$: 节点 x 到节点 y 的最低费用路径的费用。用 Bellman-Ford 方程表示

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\} \quad (4-1)$$

- ✓ $c(x, v) + d_v(y)$: x 与某个邻居 v 之间的直接链路费用 $c(x, v)$ 加上邻居 v 到 y 的最小费用。即 x 经 v 到节点 y 的最小的路径费用。
- ✓ $\min_v$: 从所有经直接相连邻居到节点 y 的费用中选取的最小路径费用。

例一如图4-25

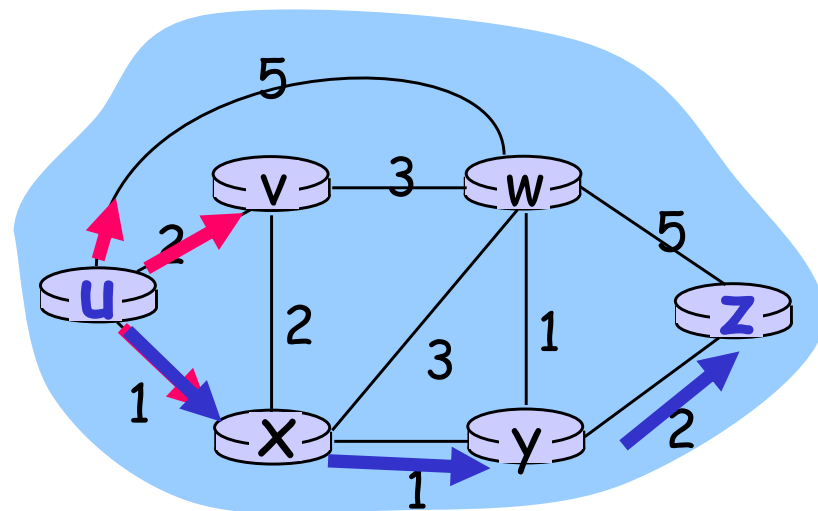
源节点u到目的节点z的最低费用路径。

源节点u有三个邻居：

- ✓ 邻居v: $d_v(z) = 5$ 、 $c(u, v) = 2$
- ✓ 邻居w: $d_w(z) = 3$ 、 $c(u, w) = 5$
- ✓ 邻居x: $d_x(z) = 3$ 、 $c(u, x) = 1$

$$\begin{aligned} d_u(z) &= \min \{ c(u, v) + d_v(z), \\ &\quad c(u, w) + \\ &\quad d_w(z) , \\ &\quad c(u, x) + \\ &\quad d_x(z) \} \\ &= \min \{ 2+5, \quad 5+3, \quad 1+3 \} \\ &= 4 \end{aligned}$$

即源节点u经相邻节点x到目的节点z的路径费用最低，为4。



得到节点u的转发表中到目的节点z的下一跳是节点x

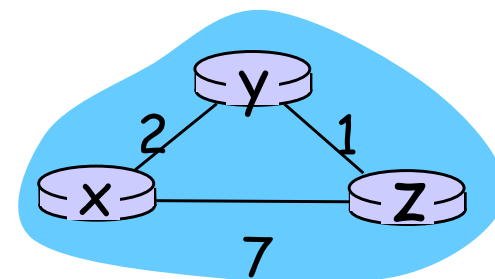
节点x选路表

- 节点x选路表（距离表）：
- ✓ 行：该节点的距离向量 D_x 和其邻居的距离向量 D_v
- ✓ 列：所有目的节点。
- 节点x的距离向量 D_x ，即节点x到每个目的节点y的估计费用； $D_x = [D_x(y): y \text{ 在 } N \text{ 中}]$
- 节点x每个邻居的距离向量 D_v ，即x的邻居v到每个目的节点y的估计费用， $D_v = [D_v(y): y \text{ 在 } N \text{ 中}]$

		x	y	z	
邻居	D_x	x	$D_x(x)$	$D_x(y)$	$D_x(z)$
	D_y	y	$D_y(x)$	$D_y(y)$	$D_y(z)$
	D_z	z	$D_z(x)$	$D_z(y)$	$D_z(z)$



	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞



更新其距离向量

- ✓ 每个节点不断向邻居发送其距离向量拷贝；
- ✓ 当节点 x 收到一个邻居 v 的新距离向量，先保存，并用公式（4-1）更新自己的距离向量：

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

从所有经邻居 v 到节点 y 的费用中选取最小路径费用。

- ✓ 若距离向量发生改变，将新的距离向量通知给邻居。

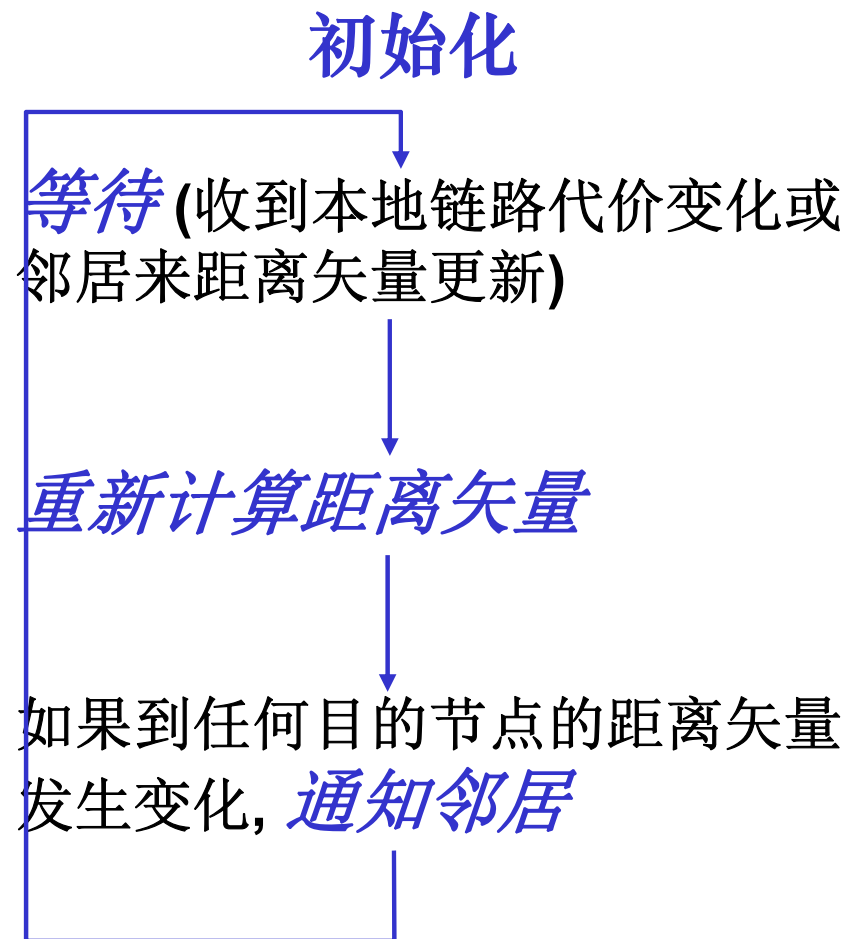
当距离向量不再变化，算法静止

此时 $D_x(y)$ 收敛到 $d_x(y)$ ，即得到节点 x 到节点 y 的最低费用路径。

距离向量DV算法描述

对每个节点x

- (1) 初始化:
- (2) 更新自己的距离向量
- (3) 重复执行 (2) , 直到没有更新的距离向量



距离矢量算法

在所有的节点, x :

初始化

```
1 Initialization:
2   for all destinations y in N:
3      $D_x(y) = c(x, y)$  /* if y is not a neighbor then  $c(x, y) = \infty$  */
4   for each neighbor w
5      $D_w(y) = \infty$  for all destinations y in N
6   for each neighbor w
7     send DV  $D_x = [D_x(y) : y \text{ in } N]$  to w

9 loop
10  wait (until I see a link cost change to neighbor w or
11        until I receive update from neighbor w)
13  for each y in N:
14     $D_x(y) = c(x, w) + D_w(y)$ 
15
16  if  $D_x(y)$  changed for any destination y
17    Send DV  $D_x = [D_x(y) : y \text{ in } N]$  to all neighbors
19 forever
```


(1) 初始化

- ✓ 计算节点 x 到所有目的点 y 的距离向量 $D_x(y)$
 - 若目的点 y 是节点 x 的邻居, 则 $D_x(y) = c(x, y)$
 - 否则, $D_x(y) = \infty$ (2#~3#)
- ✓ 节点 x 的每一个邻居 w 到所有目的点 y 的距离向量为 $D_w(y) = \infty$ (4#~5#)
- ✓ 把节点 x 的距离向量 $D_x = [D_x(y): y \text{ 在 } N \text{ 中}]$ (节点 x 到每个目的节点 y 的估计费用) 发给每一个邻居 w (6#~

```
1 Initialization:
2   for all destinations y in N:
3        $D_x(y) = c(x, y)$  /* if y is not a neighbor then  $c(x, y) = \infty$  */
4   for each neighbor w
5        $D_w(y) = \infty$  for all destinations y in N
6   for each neighbor w
7       send DV  $D_x = [D_x(y) : y \text{ in } N]$  to w
```

(2) 更新距离向量

发现直接相连的链路费用变化，或收到邻居的新距离向量，更新自己的距离向量 (10#~11#)

- ✓ 用邻居的新距离向量更新到所有目的点 y 的距离向量 (13#~14#)

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

在经每个邻居 v 到目的点 y 的距离向量中，选择一个最小值作为当前新距离向量，并将所经过的邻居 v 作为节点 x 转发表中到目的节点 y 的下一跳路由器 $v^*(y)$ 。

- ✓ 若距离向量发生改变，向其邻居发送新距离向量。 (16#~17#)

例图4-27

每个节点维护一张**选路表（距离表）**，随着算法进行不断变化。

- ✓ **行**：一个**距离向量**，每行分别表示**该节点的距离向量**和**其邻居的距离向量**，即该节点和邻居到所有目的节点的估计费用。
- ✓ **列**：所有目的节点。

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{2+0, 7+1\} = 2$$

直接到y费用最小。x到y的下一跳为y

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min\{2+1, 7+0\} = 3$$

x经y到z费用最小。x到z的下一跳为y

节点x的邻居表

目的节点

邻居

	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

y 的新距离向量

	y	z
y	∞	∞
z	∞	∞

z 的新距离向量

	y	z
y	2	0
z	∞	∞

节点 z

向邻居发送

	y	z
x	∞	∞
y	∞	∞
z	7	1

初始化

	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

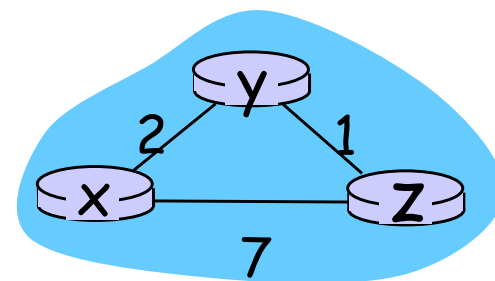
	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

稳定



第一次

第二次

time

DV算法基本思想

设

- ✓ $D_x(y)$: 节点 x 到 N 中某个节点 y 的估计费用;
- ✓ D_x : 节点 x 的距离向量。 $D_x = [D_x(y): y \text{ 在 } N \text{ 中}]$, 即节点 x 到 N 中所有其他节点 y 的估计费用。

r 基本思想:

每个节点有一张选路表（距离表），维持选路数据，随着算法进行，不断更新，直到静止。

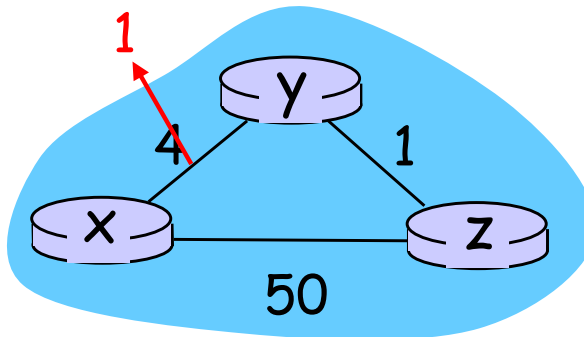
- ✓ 节点 x 选路表
- ✓ 更新选路表的距离向量
- ✓ 当距离向量不再变化，算法静止

2、链路费用改变与链路故障

当一个节点检测到**从它到邻居的链路费用发生变化时**，就更新其距离向量，如果最低费用路径的费用发生变化，通知其邻居。

(1) 某链路费用减少时情况

例图4-28。当**y 到x**的链路费用从**4变为1**的情况。

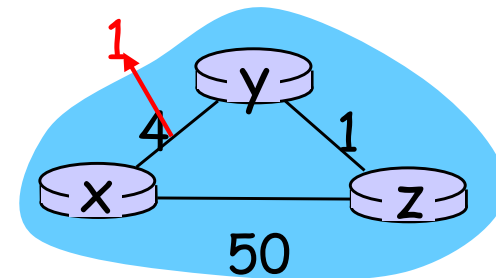


考虑y与z到目的节点x的距离表变化

- ✓ **t0**: y 检测到x的链路费用从**4变为1**，更新其距离向量，并通知其邻居z；
- ✓ **t1**: z收到来自y的更新报文，并更新自己的距离表，此时到节点x的**最低费用减为2**，并通知其邻居y；
- ✓ **t2**: y收到来自z的更新报文，并更新自己的距离表，此时到节点x的**最低费用不变仍为1**。不发送更新报文，算法静止。

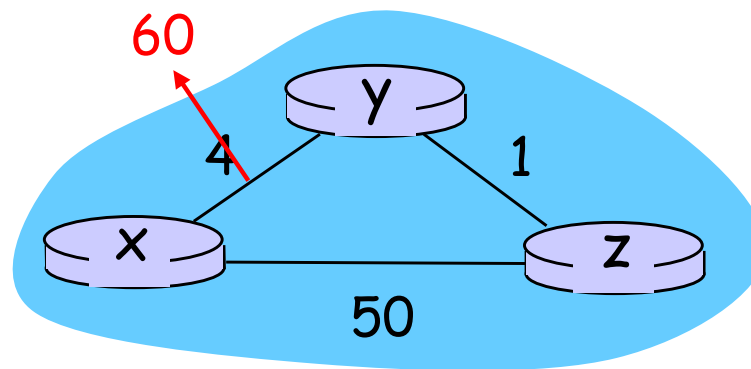
当x与y之间费用减少，DV算法**只需两次迭代**到达静止状态。

节点之间链路费用减少的“好消息”在网络中能迅速传播。



(2) 某链路费用增加时情况

例图4-28b，设x与y之间的链路费用从4增加到60。



r 链路费用变化前

$$Dy(x) = 4, \quad Dy(z) = 1, \quad Dz(y) = 1, \quad Dz(x) = 5$$

- ✓ **t0 时刻:** y检测到链路费用从4变为60。更新到x的最低路径费用

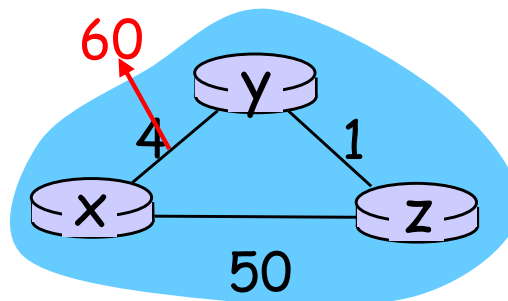
$$\begin{aligned} Dy(x) &= \min\{c(y, x) + Dx(x), c(y, z) + Dz(x)\} \\ &= \min\{60 + 0, 1 + 5\} = 6 \end{aligned}$$

y经节点z到x费用最低，此新费用错误，发给节点z。

- ✓ **t1 时刻:** z收到新费用，更新其到x的最低路径费用

$$\begin{aligned} Dz(x) &= \min\{c(z, x) + Dx(x), c(z, y) + Dy(x)\} \\ &= \min\{50 + 0, 1 + 6\} = 7 \end{aligned}$$

z经节点y到x费用最低，发给节点y。



✓ **t2 时刻:** y收到新费用, 更新到x的最低路径费用

$$D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} \\ = \min\{60 + 0, 1 + 7\} = 8$$

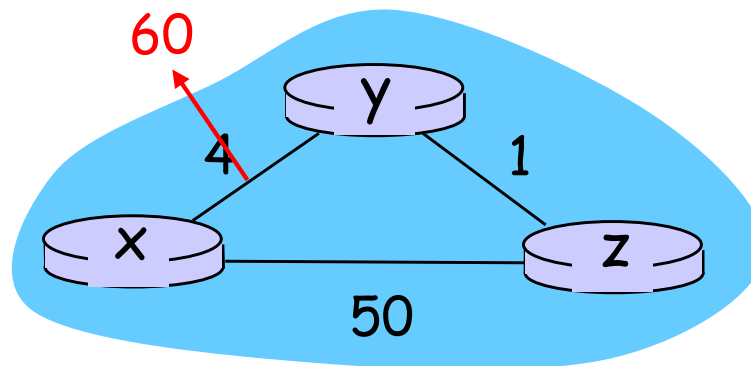
y经节点z到x费用最低, 发给节点z。

.....

节点y或z的最低费用不断更新。

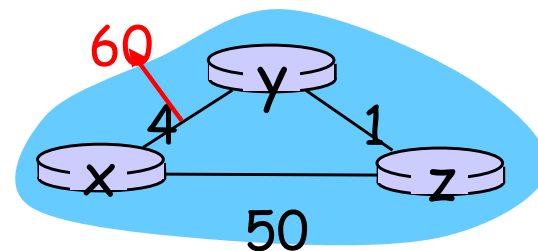
r 产生“选路回环”：为到达x, y通过z选路, z又通过y选路。

即目的地为x的分组到达y或z后, 将在这两个节点之间不停地来回反复, 直到转发表发生改变为止。



上述循环将持续44次迭代 (y与z之间的报文交换), 直到z最终算出它经由y的路径费用大于50为止。并确定:

- ✓ z到x的最低费用路径: **zx**
- ✓ y到x的最低费用路径: **yzx**



r 说明: **链路费用增加的“坏消息”传播很慢!**

当链路费用增加很大, 会出现“计数到无穷”问题。
如链路费用 $c(y, x)$ 变为10000, $c(z, x)$ 变为9999时。

3、增加“毒性逆转”

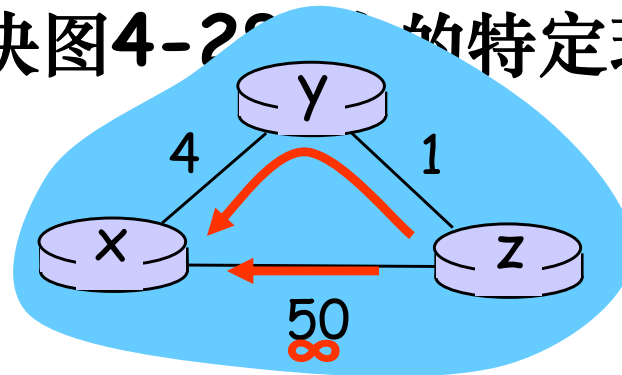
避免出现“选路回环”现象。

基本思想：

如果 z 通过 y 选路到达目的地 x ，则 z 将告诉 y ，它到 x 的距离是无穷大（即 z 将告诉 y ， $Dz(x) = \infty$ ）；

y 认为 z 没有到 x 的路径，就不会试图经由 z 选路到 x 。

例，“毒性逆转”用来解决图4-22中的特定环路



设使用“毒性逆转”后，y距离表显示 $Dz(x) = \infty$ 。

- ✓ **t0 时刻:** (x,y)链路费用从4变为60。y更新其到x的最低费用

$$\begin{aligned} Dy(x) &= \min\{c(y, x) + Dx(x), c(y, z) + Dz(x)\} \\ &= \min\{60 + 0, 1 + \infty\} = 60 \end{aligned}$$

y直接选路到x，将新费用 $Dy(x) = 60$ ，通知z。

- ✓ **t1 时刻:** z收到新费用，计算其到x的新的最低路径费用

$$\begin{aligned} Dz(x) &= \min\{c(z, x) + Dx(x), c(z, y) + Dy(x)\} \\ &= \min\{50 + 0, 1 + 60\} = 50 \end{aligned}$$

z直接选路到x，将新费用 $Dz(x) = 50$ ，通知y。

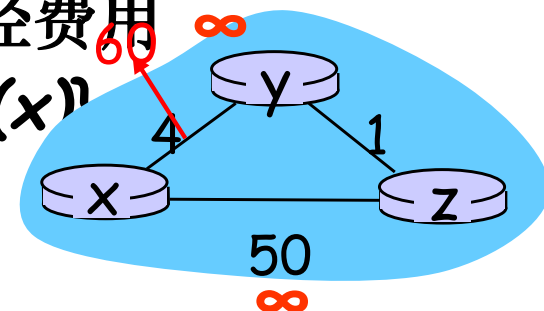
- ✓ **t2 时刻:** y收到新费用，计算其到x的新的最低路径费用

$$\begin{aligned} Dy(x) &= \min\{c(y, x) + Dx(x), c(y, z) + Dz(x)\} \\ &= \min\{60 + 0, 1 + 50\} = 51 \end{aligned}$$

y经z到x费用最低，使用“毒性逆转”抑制从y到x方向的路径

- ✓ **t3 时刻:** y通知z，y到x费用无穷大， $Dy(x) = \infty$ 。

算法静止。



毒性逆转局限

- ✓ 未解决“不可计数问题”。
- ✓ 当涉及到三个或更多节点的回路将不能被毒性逆转技术检测到。

4、LS算法与DV算法比较

- ✓ **DV算法**: 每个节点**只与邻居互相交流**, 得到邻居的新费用, 并告知邻居自己的当前最低费用。
- ✓ **LS算法**: 每个节点**与所有其他节点广播交流**, 只告知与其直接相连链路的费用。
- r **报文复杂性**:
 - ✓ **LS算法**: 知道网络每条链路的费用, 需发送 $O(nE)$ 个报文; **当一条链路的费用变化时, 必须通知所有节点**
 - ✓ **DV算法**: 迭代时, **在两个直接相连邻居之间交换报文**; 收敛时间受许多因素影响; 当链路费用改变时, **只有该链路相连的节点的最低费用路径发生改变时, 才传播已改变的链路费用**。

r 收敛速度:

- ✓ **LS算法**: 需要 $O(nE)$ 个报文和 $O(n^2)$ 的搜寻。
- ✓ **DV算法**: 收敛较慢。可能会遇到选路回环等。

r **健壮性:**

当一台路由器发生故障、操作错误或受到破坏时，会发生什么情况？

- ✓ **LS算法:** 路由器向其连接的一条链路广播不正确费用。路由计算基本独立（仅计算自己的转发表），有一定健壮性。
- ✓ **DV算法:** 一个节点可向任意或所有目的节点发布其不正确的最低费用路径。

一个节点的计算值会传递给它的邻居，并间接地传递给邻居的邻居。

一个不正确的计算值会扩散到整个网络。

5、其他选路算法

r 热土豆选路算法(hot potato routing):

路由器**尽可能快**地转发输出分组。

向任意不拥塞的输出链路转发分组，不管目的地在哪里。

r 网络流算法:

把分组流量看成是一个网络中源和目的地之间的流。将选路问题形式化为**数学上的受限优化问题**（网络流问题）。

r 电路交换选路算法:

每条链路资源(如缓冲区、部分链路带宽)都要保留给每个要经过该链路的连接。

5.3 & 5.4 自治系统内部和外部路由

前面的选路方法，需要网络的**所有路由器执行相同的选路算法**来计算通过整个网络的路径。

r 带来问题：

- ✓ 规模
- ✓ 管理自治

r 自治系统：

按区域或自治系统的形式组织路由器。**将一个大的系统划分成若干小系统（自治系统），自治系统之间再互连。**

r 自治系统AS

按**区域划分**的系统。每个**AS**由一组在**相同管理者控制下**的路由器组成。

同一个**AS**内的路由器可运行相同的选路算法，且拥有相互之间的信息。

- ✓ **自治系统内部的路由选择OSPF开放最短路径优先 (5.3)**

在**自治系统内**运行的选路算法。

- ✓ **ISP之间的路由选择BGP 边界网关协议 (5.4)**

互连各AS，负责转发目的地在本**AS**之外的分组（将本**AS**内的分组转发到另一个**AS**的路由器）。

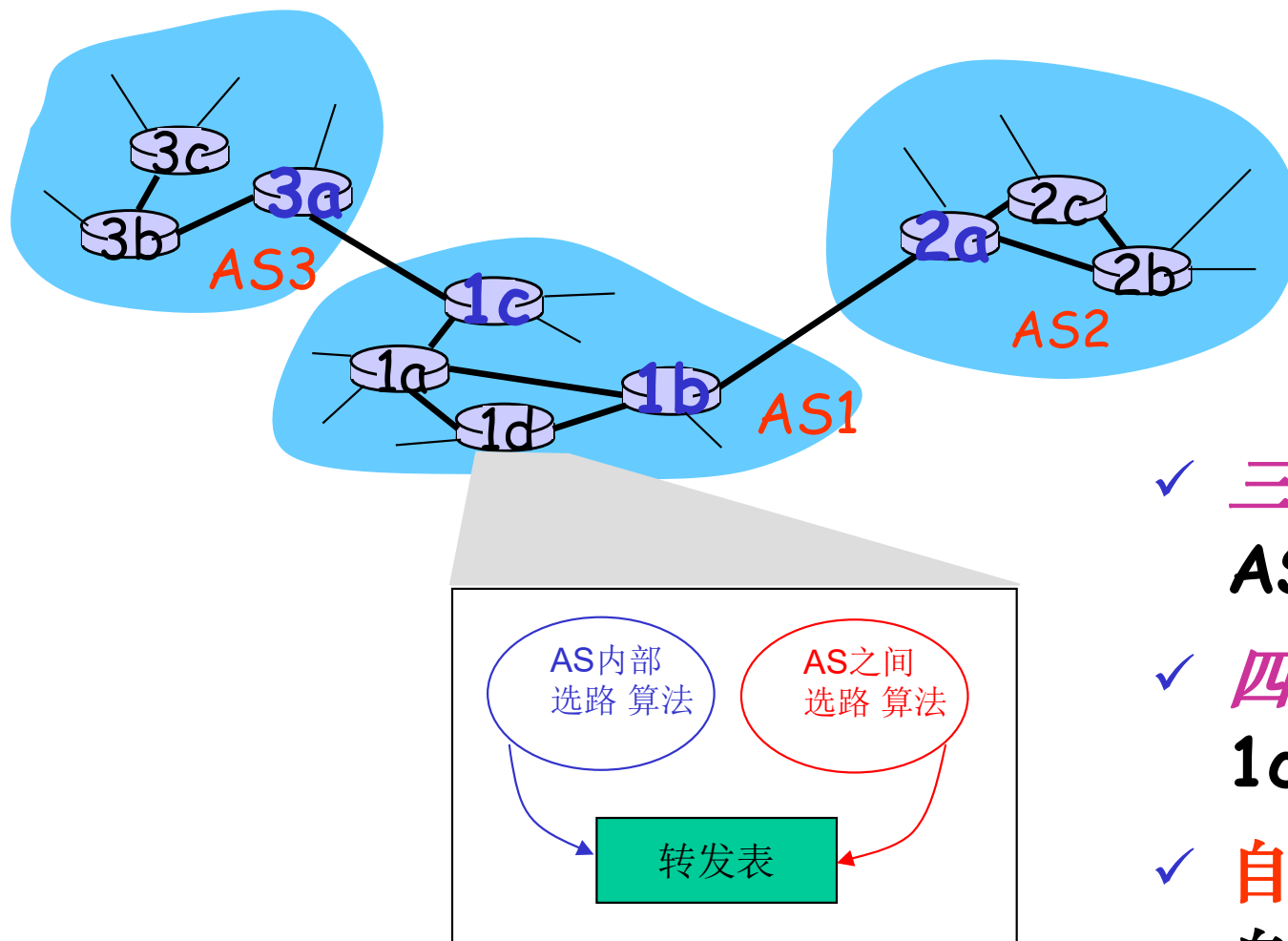
✓ 自治系统间选路协议:

在各**AS之间**进行选路的选路算法。将分组从一个**AS**选路到另一个**AS**。

✓ 路由器转发表:

由**AS**内部选路协议和**AS**间选路协议产生。

例AS间的互联



- ✓ 三个自治系统：AS1、AS2和AS3。
- ✓ 四个网关路由器：1b、1c、2a、3a。
- ✓ 自治系统内分别运行各自的内部选路协议；
- ✓ 自治系统间运行AS间选路协议。

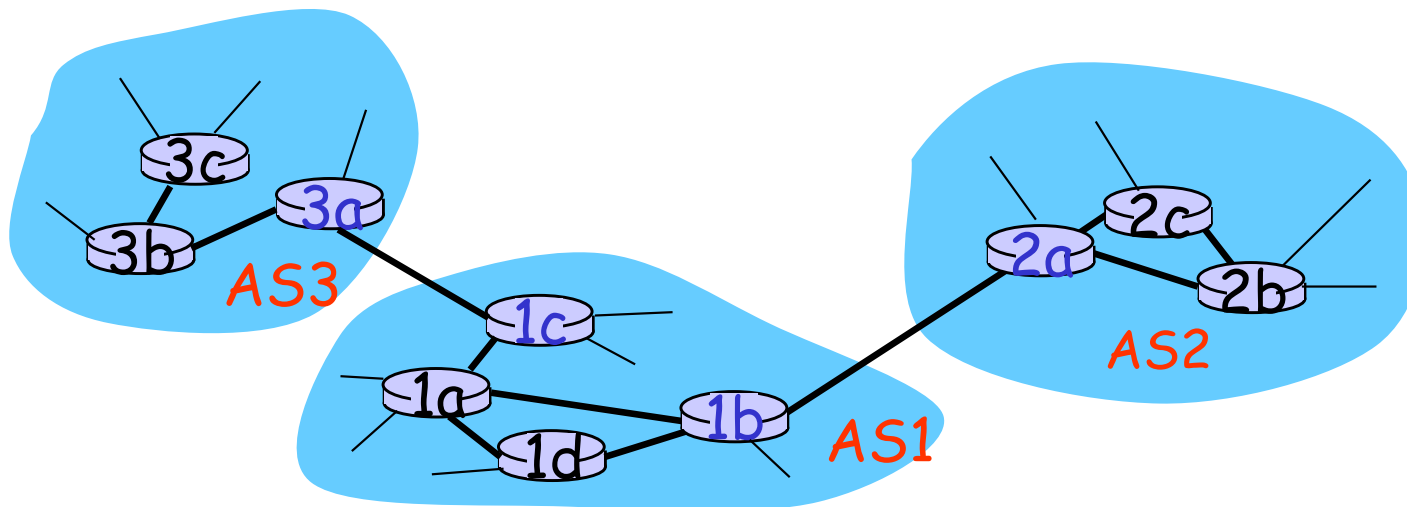
域间路由的任务

- r 假定**AS1**中的路由器接收目的地是**AS1**外部的数据报
 - m 路由器应当将分组朝着哪一个网关路由器转发?

AS1需要:

1. 知道通过**AS2**可到达哪些目的地, 通过**AS3**到达哪些
2. 传播这些可达信息到**AS1**中所有路由器

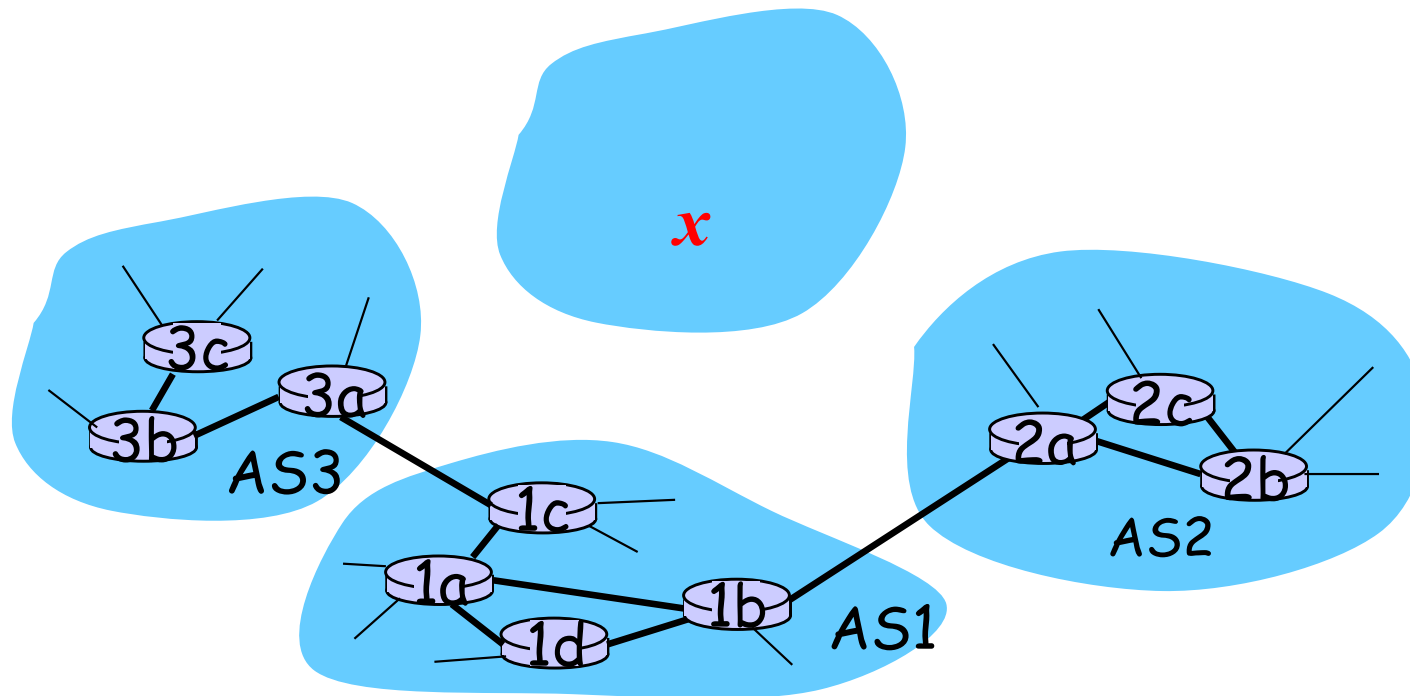
AS间选路的工作!



例子：路由器1d的转发表

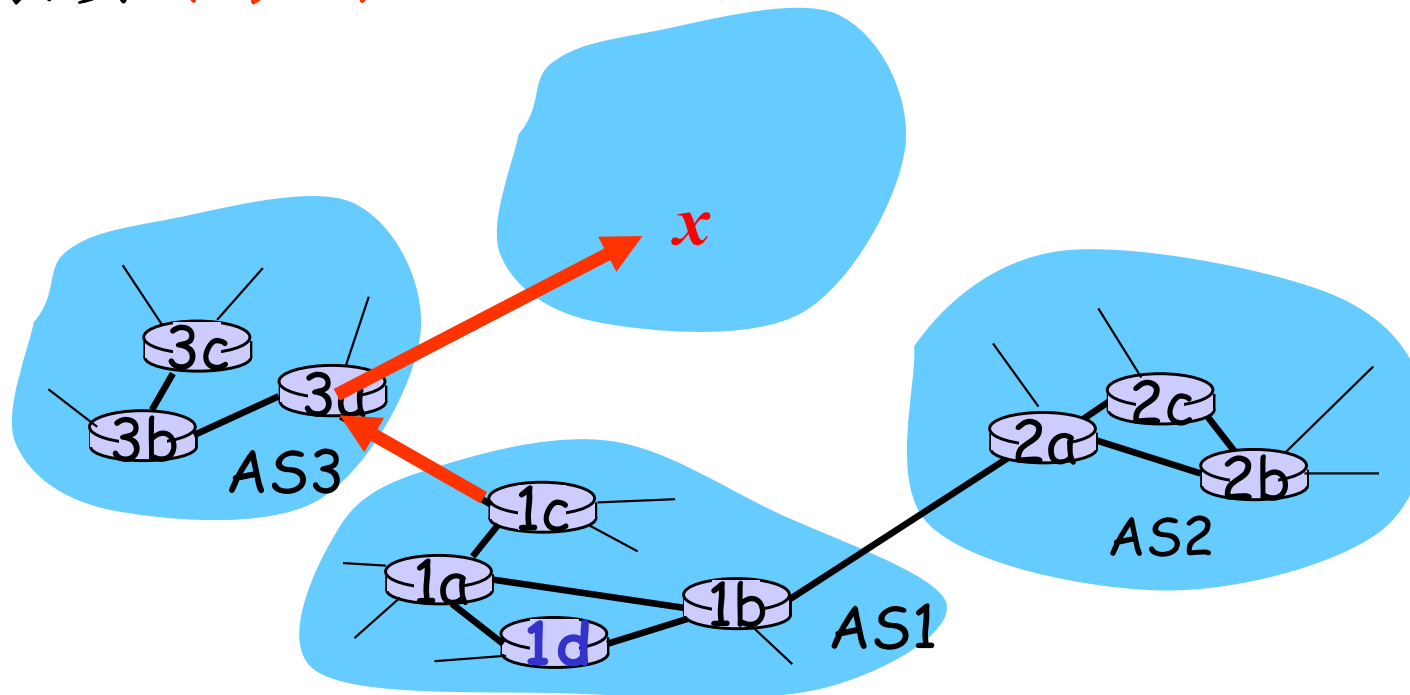
设一个子网x，AS1运行AS间选路协议后，得出：

- ✓ 子网x只经AS3到达
- ✓ 子网x可以经AS3或AS2到达



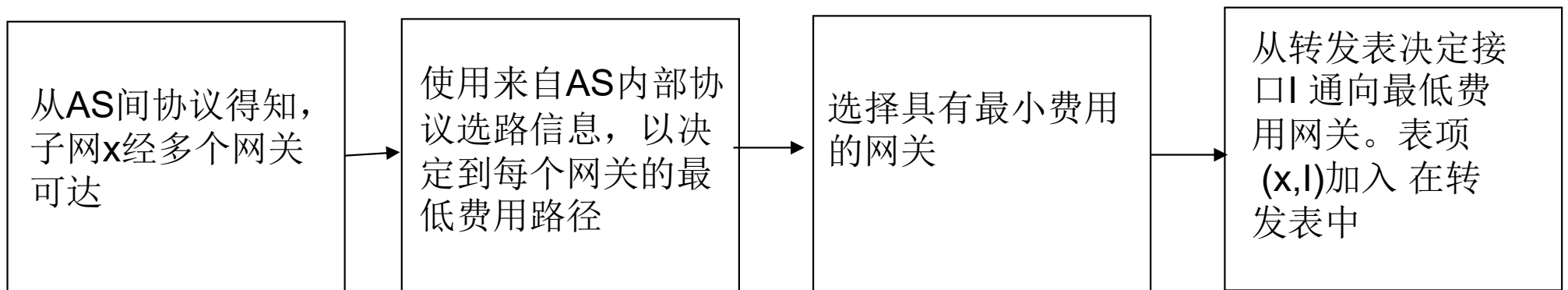
子网x只能经网关1c从AS3到达

- ✓ AS1向其所有路由器传播该信息；
- ✓ 路由器1d收到后，通过AS内部选路协议决定到网关1c的最低费用路径，并确定从接口I转发，在其转发表增加表项 (x, I) 。



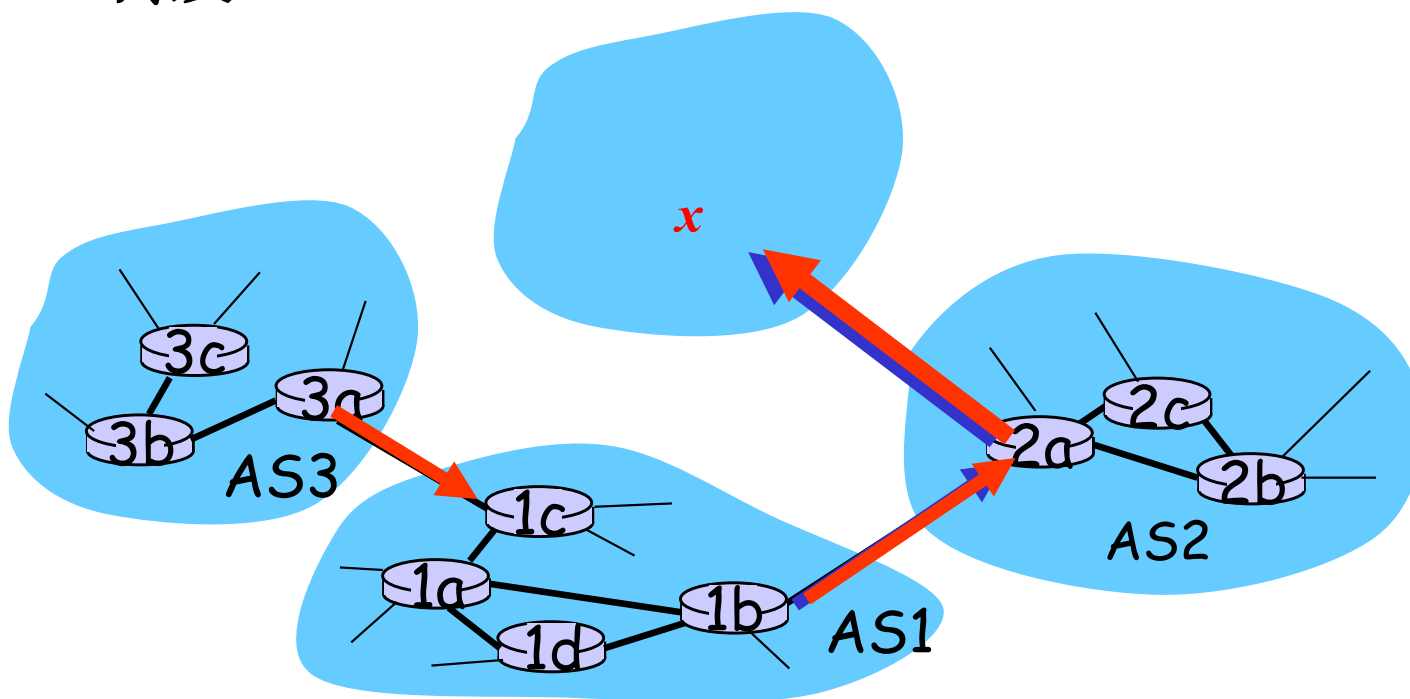
在多个AS之间选择

- 子网x可以经网关1c从AS3到达，也可以经网关1b从AS2到达
- ✓ AS1向其所有路由器传播该信息；
- ✓ 路由器1d收到后，通过AS内部选路协议决定到目的地具有最低费用的网关1c或1b，若确定从接口I转发，在其转发表增加表项 (x, I)。

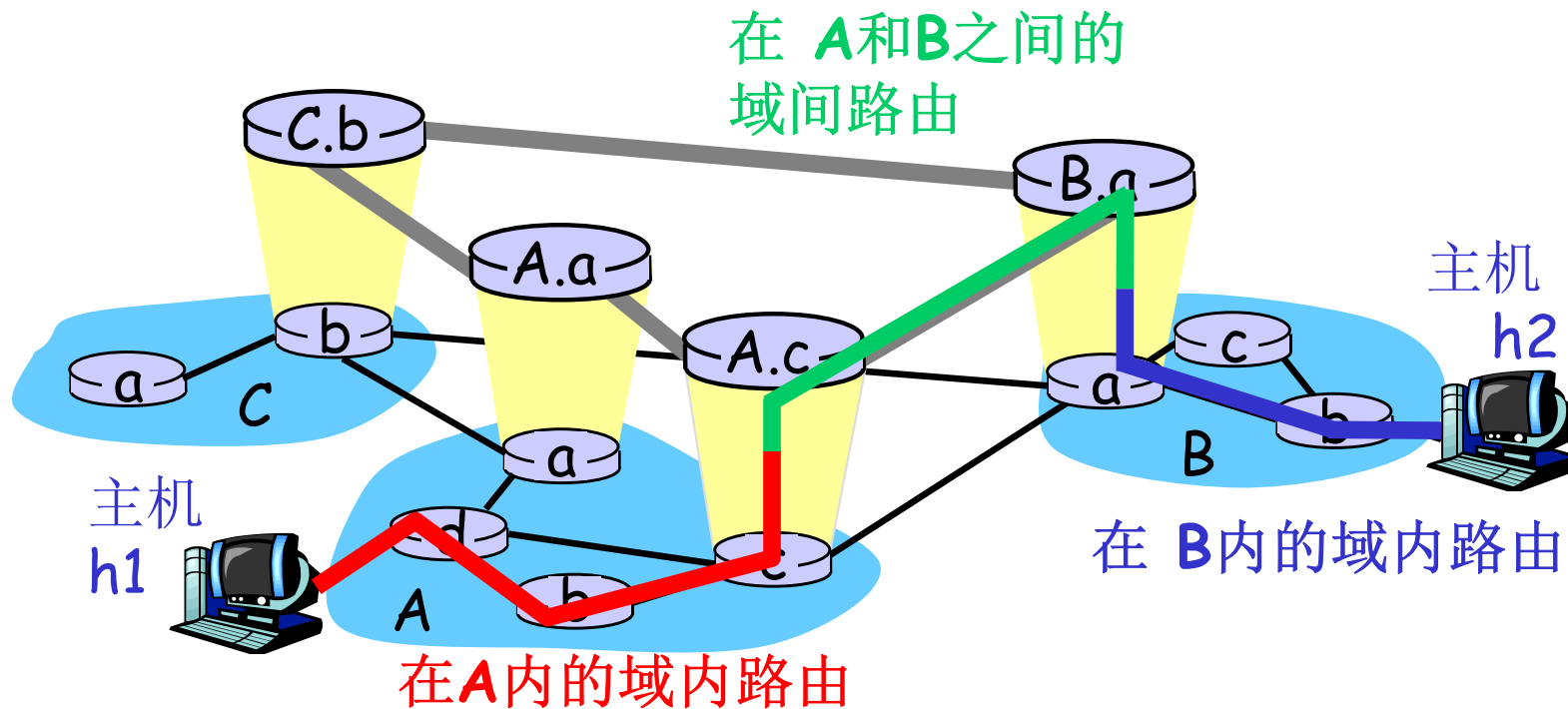


AS间选路信息的交换

- 当一个AS知道从一个相邻的AS可以到达目的地，**向所有其他的相邻AS通告该选路信息。**
- 如，AS1从AS2处得知，经AS2可以到达子网x，AS1告诉AS3，经AS1可以到达子网x。
- 如果AS3向目的地x发送分组，先向AS1转发，AS1再向AS2转发。



例：域内路由和域间路由



主机**h1**向主机**h2**发送

自治系统优点

- ✓ **减少规模大的网络选路计算的复杂性**

AS内部路由器运行相同的自治系统内部选路协议，仅需要知道本**AS**内的路由器与网关路由器。

各**AS**之间，运行相同的**AS**间选路协议。

- ✓ **管理职权灵活**

一个组织可自行选择**AS**内部选路协议，每对相连的**AS**运行相同**AS**间选路协议，交换信息。

因特网的层次选路

r **AS内部选路协议 (5.3)**

- ✓ **RIP选路信息协议**: 基于距离向量的路由协议。
- ✓ **OSPF开放最短路径优先**: 采用Dijkstra最短费用路径算法, 是一种链路状态协议。

r **AS间选路协议 (5.4)**

- ✓ **BGP边界网关协议**: 相邻BGP路由器相互交换路径信息。
- ✓ Border Gateway Protocol

RIP协议（补充）

- (Routing Information Protocol)
- 从一路由器到直接连接的网络的距离定义为 1
- “距离”也称为“跳数” (hop count)，每经过一个路由器，跳数就加 1
- 允许一条路径最多只能包含 15 个路由器。“距离”为16 时即不可达。RIP 只适用于小型互联网
- 按固定的时间间隔交换路由信息。交换的信息是当前本路由器所知道的全部信息，即自己的路由表
- RIP 选择一个具有最少路由器的路由（即最短路由）

OSPF协议 P255

- (Open Shortest Path First)开放最短路径优先
- 为了用于规模很大的网络，OSPF将一个自治系统再划分为若干个更小区域，好处：将利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个的自治系统，减少整个网络上的通信量
- 在一个区域内部的路由器只知道本区域的完整网络拓扑，而不知道其他区域的网络拓扑的情况
- 当链路状态发生变化时，路由器才用洪泛法向本区域所有路由器发送链路变化信息

BGP协议 P256

Border Gateway Protocol边界网关协议

BGP为每个AS提供了进行以下工作的手段

- 1) 从相邻AS处获得子网的可达性信息
- 2) 向本AS内部的所有路由器传播这些可达性信息
- 3) 基于可行性信息和AS策略，决定到达子网的“好”路由

- **BGP-4** 提供了一套新的机制以支持无类域间路由。
- 这些机制包括支持网络前缀的通告、取消 **BGP** 网络中“类”的概念。
- **BGP-4** 引入机制支持路由聚合，包括 AS 路径的集合。
- **BGP-4** 采用了路由向量路由协议，在配置**BGP**时，每一个自治系统的管理员要选择至少一个路由器作为该自治系统的“**BGP**发言人”。

BGP对于相同前缀存在两条或多条路由时，顺序地调用下列消除规则，直到留下一条路由。

- 1) 路由被指派一个本地偏好值作为它们的属性之一。
- 2) 在余下的路由中（所有路由都有相同的本地偏好值），具有最短**AS-PATH**的路由将被选择（由**DV**算法确定路径，使用**AS**跳数）
- 3) 在余下的路由中（所有都具有相同的本地偏好和相同的**AS-PATH**长度），选择最靠近**Next-Hop**路由器的路由。
- 这里最靠近是指具有最低费用路径的费用的路由。

小结

✓ **选路算法：链路状态和距离矢量**

谢谢大家！

Chapter 5

Network Layer:

The Control Plane

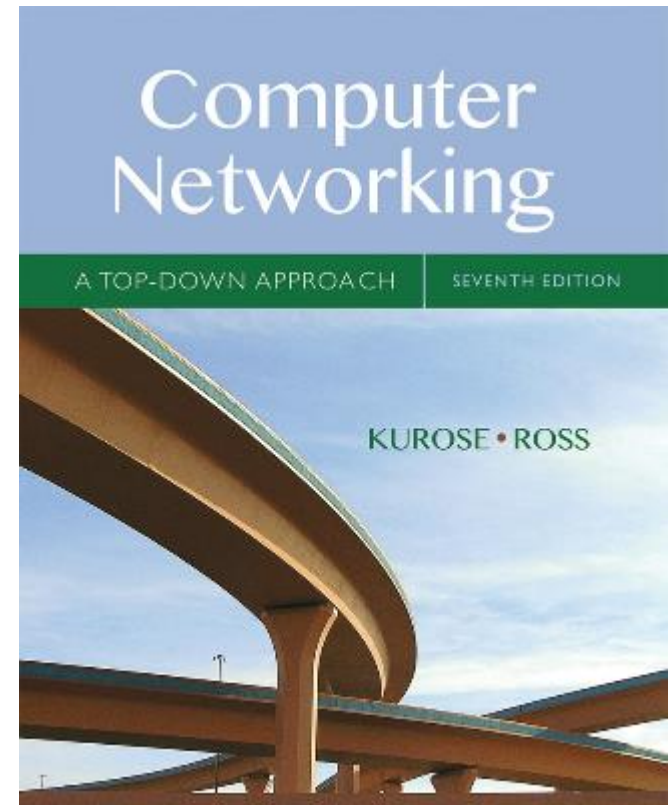
A note on the use of these Powerpoint slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

© All material copyright 1996-2016
J.F Kurose and K.W. Ross, All Rights Reserved



Computer Networking: A Top Down Approach

7th edition

Jim Kurose, Keith Ross

Pearson/Addison Wesley

April 2016

Network Layer: Control Plane

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

SDN



408考研计算机网络部分2022年新增知识点 ——软件定义网络SDN

湖科大教书匠
jgao1@hnust.edu.cn



ICMP: internet control message protocol

- used by hosts & routers to communicate network-level information

- error reporting:
unreachable host, network, port, protocol
- echo request/reply (used by ping)

- network-layer “above” IP:

- ICMP msgs carried in IP datagrams

- **ICMP message:** type, code plus first 8 bytes of IP datagram causing error

<u>Type</u>	<u>Code</u>	<u>description</u>
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Internet Control Message Protocol

Type: 8 (Echo (ping) request)

Code: 0

Traceroute and ICMP

- source sends series of UDP segments to destination
 - first set has TTL = 1
 - second set has TTL=2, etc.
 - unlikely port number
- when datagram in n th set arrives to n th router:
 - router **discards datagram and sends source ICMP message (type 11, code 0)**
 - ICMP message include name of router & IP address

```
▼ Internet Control Message Protocol
  Type: 11 (Time-to-live exceeded)
  Code: 0 (Time to live exceeded in transit)
  Checksum: 0xf4ff [correct]
```

- when ICMP message arrives, source records RTTs

stopping criteria:

- UDP segment eventually arrives at destination host
- destination returns ICMP “port unreachable” message (**type 3, code 3**)
- source stops

```
▼ Internet Control Message Protocol
  Type: 3 (Destination unreachable)
  Code: 3 (Port unreachable)
  Checksum: 0x1c38 [correct]
  Unused: 00000000
  > Internet Protocol Version 4, Src: 192.168.1.106,
  > User Datagram Protocol, Src Port: 137 (137), Dst
```

Ping ip -n 3 -l 5000

```
C:\Users\Min>ping 192.168.1.1 -n 3 -l 5000
```

正在 Ping 192.168.1.1 具有 5000 字节的数据:

来自 192.168.1.1 的回复: 字节=5000 时间=2ms TTL=64

来自 192.168.1.1 的回复: 字节=5000 时间=2ms TTL=64

来自 192.168.1.1 的回复: 字节=5000 时间=3ms TTL=64

192.168.1.1 的 Ping 统计信息:

数据包: 已发送 = 3, 已接收 = 3, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):

最短 = 2ms, 最长 = 3ms, 平均 = 2ms

icmp

Expressi

No.	Time	Source	Destination	Protocol	Length	Info
← 47	3.921507	192.168.1.1	192.168.1.106	ICMP	602	Echo (ping) reply id=0x0001, seq=119/30464, ttl=64 (request in 43)
60	4.928785	192.168.1.1	192.168.1.106	ICMP	602	Echo (ping) reply id=0x0001, seq=120/30720, ttl=64 (request in 56)
68	5.936325	192.168.1.1	192.168.1.106	ICMP	602	Echo (ping) reply id=0x0001, seq=121/30976, ttl=64 (request in 64)
→ 43	3.918951	192.168.1.106	192.168.1.1	ICMP	602	Echo (ping) request id=0x0001, seq=119/30464, ttl=64 (reply in 47)
56	4.923920	192.168.1.106	192.168.1.1	ICMP	602	Echo (ping) request id=0x0001, seq=120/30720, ttl=64 (reply in 60)
64	5.931062	192.168.1.106	192.168.1.1	ICMP	602	Echo (ping) request id=0x0001, seq=121/30976, ttl=64 (reply in 68)

Destination: 192.168.1.1
 [Source GeoIP: Unknown]
 [Destination GeoIP: Unknown]

▼ [4 IPv4 Fragments (5008 bytes): #40(1480), #41(1480), #42(1480), #43(568)]

[Frame: 40, payload: 0-1479 (1480 bytes)]
 [Frame: 41, payload: 1480-2959 (1480 bytes)]
 [Frame: 42, payload: 2960-4439 (1480 bytes)]
 [Frame: 43, payload: 4440-5007 (568 bytes)]
 [Fragment count: 4]
 [Reassembled IPv4 length: 5008]
 [Reassembled IPv4 data: 080040d8000100776162636465666768696a6b6c6d6e6f70...]

▼ Internet Control Message Protocol

Type: 8 (Echo (ping) request)
 Code: 0
 Checksum: 0x40d8 [correct]
 Identifier (BE): 1 (0x0001)
 Identifier (LE): 256 (0x0100)
 Sequence number (BE): 119 (0x0077)
 Sequence number (LE): 30464 (0x7700)
 [Response frame: 47]

▼ Data (5000 bytes)

Data: 6162636465666768696a6b6c6d6e6f707172737475767761...
 [Length: 5000]



图6-1 ICMP报文格式

C:\WINDOWS\system32\cmd.exe

Microsoft Windows [版本 10.0.18362.720]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Users\Min>tracert www.qq.com

通过最多 30 个跃点跟踪
到 public-v6.sparta.mig.tencent-cloud.net [157.255.192.44] 的路由:

1	1 ms	1 ms	1 ms	bogon [192.168.1.1]
2	5 ms	4 ms	3 ms	113.205.28.1
3	*	31 ms	6 ms	221.5.230.77
4	5 ms	6 ms	7 ms	221.5.253.26
5	5 ms	5 ms	5 ms	221.5.239.53
6	42 ms	32 ms	32 ms	219.158.114.113
7	39 ms	39 ms	35 ms	120.86.0.58
8	^C			

C:\Users\Min>

icmp						
No.	Time	Source	Destination	Protocol	Length	Info
4	0.101569	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=122/31232, ttl=1 (no r
5	0.103233	192.168.1.1	192.168.1.106	ICMP	126	Time-to-live exceeded (Time to live exceeded in transit)
6	0.103649	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=123/31488, ttl=1 (no r
7	0.105110	192.168.1.1	192.168.1.106	ICMP	126	Time-to-live exceeded (Time to live exceeded in transit)
8	0.105474	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=124/31744, ttl=1 (no r
9	0.107139	192.168.1.1	192.168.1.106	ICMP	126	Time-to-live exceeded (Time to live exceeded in transit)
11	1.112328	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=125/32000, ttl=2 (no r
12	1.117309	113.205.28.1	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
13	1.120346	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=126/32256, ttl=2 (no r
14	1.124146	113.205.28.1	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
15	1.126695	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=127/32512, ttl=2 (no r
16	1.130351	113.205.28.1	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
20	1.147567	113.205.28.1	192.168.1.106	ICMP	70	Destination unreachable (Port unreachable)
43	4.149932	113.205.28.1	192.168.1.106	ICMP	70	Destination unreachable (Port unreachable)
62	7.150265	113.205.28.1	192.168.1.106	ICMP	70	Destination unreachable (Port unreachable)
78	11.147476	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=128/32768, ttl=3 (no r
102	14.812019	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=129/33024, ttl=3 (no r
103	14.843226	221.5.230.77	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
104	14.846218	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=130/33280, ttl=3 (no r
105	14.852501	221.5.230.77	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
153	24.867878	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=131/33536, ttl=4 (no r
154	24.873202	221.5.253.26	192.168.1.106	ICMP	310	Time-to-live exceeded (Time to live exceeded in transit)
155	24.876280	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=132/33792, ttl=4 (no r
156	24.882693	221.5.253.26	192.168.1.106	ICMP	310	Time-to-live exceeded (Time to live exceeded in transit)
157	24.885754	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=133/34048, ttl=4 (no r
158	24.893158	221.5.253.26	192.168.1.106	ICMP	310	Time-to-live exceeded (Time to live exceeded in transit)
210	34.905117	192.168.1.106	157.255.192.44	ICMP	106	Echo (ping) request id=0x0001, seq=134/34304, ttl=5 (no r
211	34.910312	221.5.239.53	192.168.1.106	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

UDP?

- Windows, 直接发 ICMP, 大部分时间抓不到UDP
- 有时候也会
- Linux下, 抓到UDP

```
▼ Internet Control Message Protocol
  Type: 3 (Destination unreachable)
  Code: 3 (Port unreachable)
  Checksum: 0x1c38 [correct]
  Unused: 00000000
  > Internet Protocol Version 4, Src: 192.168.1.106, Dst: 113.205.28.1
  > User Datagram Protocol, Src Port: 137 (137), Dst Port: 137 (137)
```

SNMP

from bilibili



谢谢大家！